

LAB_Activity_CreateAConditionalStatement

May 3, 2026

1 Activity: Create a conditional statement

1.1 Introduction

Conditional statements are a powerful structure that help in achieving automation when you need to make sure conditions are met before certain actions are executed. For example, security analysts can use conditional statements in Python to check if users are approved to access a device.

In this lab, you will practice writing conditional statements in Python.

Tips for completing this lab

As you navigate this lab, keep the following tips in mind:

- **### YOUR CODE HERE ###** indicates where you should write code. Be sure to replace this with your own code before running the code cell.
- Feel free to open the hints for additional guidance as you work on each task.
- To enter your answer to a question, double-click the markdown cell to edit. Be sure to replace the “[Double-click to enter your responses here.]” with your own answer.
- You can save your work manually by clicking File and then Save in the menu bar at the top of the notebook.
- You can download your work locally by clicking File and then Download and then specifying your preferred file format in the menu bar at the top of the notebook.

1.2 Scenario

You’re working as a security analyst. First, you are responsible for checking whether a user’s operating system requires an update. Then, you need to investigate login attempts to a specific device. You must determine if login attempts were made by users approved to access this device and if the login attempts occurred during organization hours.

1.3 Task 1

You are asked to help automate the process of checking whether a user’s operating system requires an update. Imagine that a user’s device can be running one of the following operating systems: OS 1, OS 2, or OS 3. While OS 2 is up-to-date, OS 1 and OS 3 are not. Your task is to check whether the user’s system is up-to-date, and if it is, display a message accordingly. To do this, complete

the conditional statement using the keyword `if`. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

```
[4]: # Assign a variable named `system` to a specific operating system, represented
      ↪ as a string
      # This variable indicates which operating system is running
      # Feel free to run this cell multiple times; each time try assigning `system`
      ↪ to different values ("OS 1", "OS 2", "OS 3") and observe the result

      system = "OS 2"

      # If OS 2 is running, then display a "no update needed" message

      if system == "OS 2":
          print("no update needed")
```

no update needed

Hint 1

Use a comparison operator that will allow you to check whether the `system` is running "OS 2".

Hint 2

Use the `==` comparison operator to check whether the `system` is running "OS 2".

1.4 Task 2

Now try assigning the `system` variable to different values ("OS 1", "OS 2", and "OS 3"), run the cell, and observe what happens. Keep the conditional statement as is. Be sure to replace the `### YOUR CODE HERE ###` with your own code.

```
[10]: # Assign `system` to a specific operating system
       # This variable represents which operating system is running
       # Feel free to run this cell multiple times; each time try assigning `system`
       ↪ to different values ("OS 1", "OS 2", "OS 3") and observe the result

       system = "OS 1"

       # If OS 2 is running, then display a "no update needed" message

       if system == "OS 2":
           print("no update needed")
```

Question 1 What happens when OS 2 is running? What happens when OS 1 is running?

When OS 2 is running “no update needed” is printed, when OS 1 is running, nothing is printed

1.5 Task 3

Nothing is displayed when the `system` is not equal to "OS 2". This is because the condition didn't evaluate to `True`.

It would be beneficial if an alternative message is provided to them when updates are needed.

In the following cell, add the appropriate keyword after the first conditional so that it will display a message that conveys that an update is needed when the `system` is not running OS 2. Be sure to replace each `### YOUR CODE HERE ###` with your own code.

Then, set the value of the `system` variable to indicate that OS 2 is running and run the cell. After observing what happens, set the value of `system` to indicate either that OS 1 is running or that OS 3 is running and run the cell.

```
[14]: # Assign `system` to a specific operating system
      # This variable represents which operating system is running

      system = "OS 2"

      # If OS 2 is running, then display a "no update needed" message
      # Otherwise, display a "update needed" message

      if system == "OS 2":
          print("no update needed")
      else:
          print("update needed")
```

no update needed

Hint 1

Use the `else` keyword.

Question 2 In this setup what happens when OS 2 is running? And what happens when OS 2 is not running?

If OS 2 is running - “no update needed is printed”, if OS2 isn't running “update needed” will be printed.

1.6 Task 4

This setup is still not ideal. If the variable `system` contains a random string or integer, the conditional above would still display `update needed`.

To improve the conditional, you will need to add the `elif` keyword. In the following cell, you will add two `elif` statements after the `if` statement, to create the final code. The first `elif` statement

will display `update needed` if `system` is `"OS 1"`. The second `elif` statement will display the same message, if `system` is `"OS 3"`. Complete the second `elif` statement, and then run the cell with the variable `system` set to a different string each time. Observe what happens when each operating system is running. Also try assigning the `system` variable to some strings other than `"OS 1"`, `"OS 2"`, and `"OS 3"` (for example `"OS 4"`).

Be sure to replace each `### YOUR CODE HERE ###` with your own code.

```
[17]: # Assign `system` to a specific operating system
      # This variable represents which operating system is running

      system = "OS 2"

      # If OS 2 is running, then display a "no update needed" message
      # Otherwise if OS 1 is running, display a "update needed" message
      # Otherwise if OS 3 is running, display a "update needed" message

      if system == "OS 2":
          print("no update needed")
      elif system == "OS 1":
          print("update needed")
      elif system == "OS 3":
          print("update needed")
```

no update needed

Question 3 Under this setup what happens when OS 2 is running? What happens when OS 1 is running? What happens when OS 3 is running? What happens when neither of those three operating systems are running?

if OS 2 is running “no update needed” will be printed, if OS 1 and 3 are running “update needed” will be printed. If none of them are running, nothing will be printed

1.7 Task 5

Writing code that is readable and concise is a best practice in programming.

The conditional above can be written more concisely.

In the following cell, use a logical operator to combine the two `elif` statements from the previous setup into one `elif` statement. Be sure to replace each `### YOUR CODE HERE ###`. Then, assign the `system` variable to a value and run the cell. Like you did in the previous task, use `"OS 1"`, `"OS 2"`, `"OS 3"`, and other strings.

```
[20]: # Assign `system` to a specific operating system
      # This variable represents which operating system is running

      system = "OS 2"
```

```

# If OS 2 is running, then display a "no update needed" message
# Otherwise if either OS 1 or OS 3 is running, display a "update needed" message

if system == "OS 2":
    print("no update needed")
elif system == "OS 3" or system == "OS 1":
    print("update needed")

```

no update needed

Hint 1

Use the `or` logical operator.

Hint 2

Use the `or` operator between two conditions that each use the `==` operator.

Question 4 What do you observe about this conditional?

using “or” combines the two conditionals into one

1.8 Task 6

Now you’ll move on to the next part of your work. You’ve been asked to investigate login attempts to a specific device. Only approved users should log on to this device.

You’ll start with two authorized users, stored in the variables `approved_user1` and `approved_user2`. You’ll need to write a conditional statement that compares those variables to a third variable, `username`. This will be the username of a specific user trying to log in. Be sure to replace each `### YOUR CODE HERE ###` with your own code.

```

[22]: # Assign `approved_user1` and `approved_user2` to usernames of approved users

approved_user1 = "elarson"
approved_user2 = "bmoreno"

# Assign `username` to the username of a specific user trying to log in

username = "bmoreno"

# If the user trying to log in is among the approved users, then display a
↪message that they are approved to access this device
# Otherwise, display a message that they do not have access to this device

if username == approved_user1 or username == approved_user2:
    print("This user has access to this device.")
else:

```

```
print("This user does not have access to this device.")
```

This user has access to this device.

Hint 1

Use the `if` keyword in the first conditional statement and the `else` keyword in the second conditional statement. Make sure both statements end with the proper syntax of a colon (:).

Hint 2

Use a comparison operator that will allow you to check whether the user trying to log in is an approved user.

Hint 3

Use the `==` comparison operator to check whether the user trying to log in is an approved user.

1.9 Task 7

The number of approved users has now expanded to five. Rather than storing each of the approved users' usernames individually, it would be more concise to store them in an allow list called `approved_list`.

The `in` operator in Python can be used to determine whether a given value is an element of a sequence. Using the `in` operator in a condition can help you check whether a specific username is part of a list of approved usernames. For example, in the code below, `username in approved_list` evaluates to `True` if the value of the `username` variable is included in `approved_list`.

Complete the code in the following cell to display the same messages that you used in the previous step. When the condition evaluates to `True`, the following message will be displayed: "This user has access to this device." When it evaluates to `False`, the following message will be displayed: "This user does not have access to this device." Then, run the cell to observe its behavior. Be sure to replace each `### YOUR CODE HERE ###` with your own code. Afterwards, reassign the `username` variable to a username that is not approved and run the cell to observe what happens.

```
[23]: # Assign `approved_list` to a list of approved usernames

approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log in

username = "bmoreno"

# If the user trying to log in is among the approved users, then display a
→message that they are approved to access this device
# Otherwise, display a message that they do not have access to this device

if username in approved_list:
```

```
print("This user has an access to this device")
else:
    print("this user has no access to this device")
```

This user has an access to this device

Hint 1

Use the `else` keyword in the second conditional statement.

Hint 2

Use the `print()` function to display messages.

Question 5 What happens when an approved user tries to log in? What happens when an unapproved user tries to log in?

[Double-click to enter your responses here.]

1.10 Task 8

Now you'll write another conditional statement. This one will use a `organization_hours` variable to check if the user logged in during specific organization hours. When that condition is met, the code should display the string "Login attempt made during organization hours.". When that condition isn't met, the code should display the string "Login attempt made outside of organization hours.".

The `organization_hours` variable will have a Boolean data type. If `organization_hours` has a Boolean value of `True`, that means the user is logged in during the specified organization hours. If `organization_hours` has a Boolean value of `False`, that means the user is not logged in during those hours.

Complete the conditional in the following cell. Be sure to replace each `### YOUR CODE HERE ###` with your own code before running the following cell.

```
[25]: # Assign `organization_hours` to a Boolean value that represents whether the
      ↪ user is trying to log in during organization hours

organization_hours = True

# If the entered `organization_hours` has a value of True, then display "Login
↪ attempt made during organization hours."
# Otherwise, display "Login attempt made outside of organization hours."

if organization_hours == True:
    print("Login attempt made during organizational hours")
else:
    print("Login attempt made outside of organizational hours")
```

Login attempt made during organizational hours

Hint 1

Use the `==` comparison operator to check whether the user is logged in during the specified organization hours. Compare `organization_hours` to the appropriate Boolean value.

Hint 2

Use the `print()` function to display messages.

Hint 3

Use the `else` keyword in the second conditional statement.

Question 6 What happens when the user logs in during organization hours? What happens when they log in outside of organization hours?

When a user makes a login attempt during organizational hours, “Login attempt was made during organizational hours” displays and when it’s outside of them it, it says “Login attempt made outside of organizational hours”

1.11 Task 9

The following cell assembles the code from the previous tasks. It includes the conditional statement that checks if a user is on the allow list and the conditional statement that checks if the user logged in during organization hours.

Run the cell below a few times. Each time, enter a different combination of values for `username` and `organization_hours` to observe how that affects the output.

```
[ ]: # Assign `approved_list` to a list of approved usernames

approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log in

username = "bmoreno"

# If the user trying to log in is among the approved users, then display a
# message that they are approved to access this device
# Otherwise, display a message that they do not have access to this device

if username in approved_list:
    print("This user has access to this device.")
else:
    print("This user does not have access to this device.")
```

```

# Assign `organization_hours` to a Boolean value that represents whether the
↪user is trying to log in during organization hours

organization_hours = True

# If the entered `organization_hours` has a value of True, then display "Login
↪attempt made during organization hours."
# Otherwise, display "Login attempt made outside of organization hours."

if organization_hours == True:
    print("Login attempt made during organization hours.")
else:
    print("Login attempt made outside of organization hours.")

```

Question 7 What happens when the user trying to log in is not among the approved users? What happens when the user trying to log in is among the approved users? What happens when the user tries to log in outside of organization hours?

[Double-click to enter your responses here.]

1.12 Task 10

You can also provide a single message about the login attempt. To do this, you can join both conditions into a single conditional statement using a logical operator. This will make the code more concise.

Examine the code in the following cell and add the missing operator that would allow for a single message. Be sure to replace each **### YOUR CODE HERE ###** with your own code before running the following cell. Then run the cell, entering different combinations of information, and observe what happens.

```

[26]: # Assign `approved_list` to a list of approved usernames

approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log in

username = "bmoreno"

# Assign `organization_hours` to a Boolean value that represents whether the
↪user is trying to log in during organization hours

organization_hours = True

```

```

# If the user is among the approved users and they are logging in during
↪organization hours, then convey that the user is logged in
# Otherwise, convey that either the username is not approved or the login
↪attempt was made outside of organization hours

if username in approved_list and organization_hours == True:
    print("Login attempt made by an approved user during organization hours.")
else:
    print("Username not approved or login attempt made outside of organization
↪hours.")

```

Login attempt made by an approved user during organization hours.

Hint 1

Use the logical operator that would allow you to check both conditions in the `if` statement (the condition that the entered username is in the approved list and the condition that the login attempt is occurring during organization hours).

Hint 2

Use the `and` logical operator to check both conditions in the `if` statement (the condition that the entered username is in the approved list and the condition that the login attempt is occurring during organization hours).

Question 8 In this setup, what happens when the user trying to log in is an approved user and doing so during organization hours? What happens when the user either is not approved or attempts to log in outside of organization hours?

When the user trying to log in is an approved user and it's during organizational hours, "login attempt made by an approved user during organizational hours" prints. If the user is not approved or its outside of organizational hours, "Username not approved or login attempt made outside of organizational hours" displays.

1.13 Conclusion

What are your key takeaways from this lab? Using `==` you can determine whether one value is equal to another